МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Разработка баз данных»

Практическая работа № 7

Студент группы *ИВБО-20-24 Дьяков Андрей Геннадьевич* _____
(подпись)

Ассистент *Шерри Георгий Даниэлевич*

_____
(подпись)

Отчет представлен «___»_____2026 г.

Москва 2026 г.

ОПТИМИЗАЦИЯ ЗАПРОСОВ, ИНДЕКСЫ И ОБСЛУЖИВАНИЕ БАЗЫ ДАННЫХ В POSTGRESQL

Цель работы: Целью данной практической работы является формирование у студентов глубоких теоретических знаний и практических навыков анализа производительности SQL-запросов, а также освоение полного спектра инструментов оптимизации в СУБД PostgreSQL.

ПОДГОТОВКА БАЗЫ ДАННЫХ

Листинг 1

```
ALTER TABLE user_logs
ADD COLUMN IF NOT EXISTS ip_address inet;

WITH
country_ids AS (
    SELECT array_agg(id ORDER BY id) AS ids
    FROM country
),
supplier_ids AS (
    SELECT array_agg(id ORDER BY id) AS ids
    FROM supplier
),
airline_ids AS (
    SELECT array_agg(id ORDER BY id) AS ids
    FROM airline
),
tour_operator_ids AS (
    SELECT array_agg(id ORDER BY id) AS ids
    FROM tour_operator
)
INSERT INTO service (
    name,
    service_type,
    country_id,
    departure_city,
    supplier_id,
    description,
    airline_id,
    tour_operator_id
)
SELECT
    'Тур №' || i,
    CASE
        WHEN i % 4 = 0 THEN 'flight'
        WHEN i % 4 = 1 THEN 'tour'
        WHEN i % 4 = 2 THEN 'hotel'
        ELSE 'insurance'
    END,
    country_ids.ids[(i % cardinality(country_ids.ids)) + 1],
    CASE
        WHEN i % 4 = 0 THEN 'Москва (MOW)'
        ELSE NULL
```

```
    END,
    supplier_ids.ids[(i % cardinality(supplier_ids.ids)) + 1],
    'Описание услуги ' || i,
    airline_ids.ids[(i % cardinality(airline_ids.ids)) + 1],
    tour_operator_ids.ids[(i % cardinality(tour_operator_ids.ids)) + 1]
FROM generate_series(1, 20000) AS i
CROSS JOIN country_ids
CROSS JOIN supplier_ids
CROSS JOIN airline_ids
CROSS JOIN tour_operator_ids;

INSERT INTO customer_order (
    created_at,
    client_id,
    employee_id,
    status_id,
    paid_at,
    total_amount
)
SELECT
    created_at,
    client_id,
    employee_id,
    status_id,
    CASE
        WHEN is_paid THEN created_at + INTERVAL '1 hour'
        ELSE NULL
    END,
    total_amount
FROM (
    SELECT
        NOW() - (random() * INTERVAL '1000 days') AS created_at,
        (i % 10) + 1 AS client_id,
        ((i * 7) % 10) + 1 AS employee_id,
        ((i * 3) % 10) + 1 AS status_id,
        (i % 3 <> 0) AS is_paid,
        round((random() * 490000 + 10000)::numeric, 2) AS total_amount
    FROM generate_series(1, 20000) AS i
) x;

INSERT INTO user_logs (
    created_at,
    user_id,
    event_data,
    ip_address
)
SELECT
    NOW() - INTERVAL '1 second' * (20000 - i),
    (i % 1000) + 1,
    jsonb_build_object(
        'event',
            CASE
                WHEN i % 5 = 0 THEN 'booking'
                WHEN i % 5 = 1 THEN 'login'
                WHEN i % 5 = 2 THEN 'view_tour'
                WHEN i % 5 = 3 THEN 'search'
                ELSE 'logout'
            END,
        'user_id', (i % 1000) + 1,
        'tour_id', (i % 20000) + 1,
        'amount', round((random() * 200000)::numeric, 2),
        'success', (random() > 0.2),
        'tags',
            CASE
```

```
                WHEN i % 3 = 0 THEN jsonb_build_array('promo', 'sale')
                WHEN i % 3 = 1 THEN jsonb_build_array('news')
                ELSE jsonb_build_array('updates', 'alert')
            END
    ),
    (
        '192.168.' ||
        ((i % 254) + 1) ||
        '.' ||
        (((i / 254) % 254) + 1)
    )::inet
FROM generate_series(1, 20000) AS i;

SELECT
    (SELECT COUNT(*) FROM service)        AS service_count,
    (SELECT COUNT(*) FROM customer_order) AS order_count,
    (SELECT COUNT(*) FROM user_logs)       AS user_logs_count;
```

123 service_count	123 order_count	123 user_logs_count
20 010	20 010	20 014

Задание №1: анализ и оптимизация с использованием B-Tree

Листинг 2

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM service WHERE name = 'Тур №15000';
```

	A-Z QUERY PLAN
1	Seq Scan on service (cost=0.00..553.12 rows=1 width=95) (actual time=1.983..2.731 rows=1.00 loops=1)
2	Filter: ((name)::text = 'Тур №15000'::text)
3	Rows Removed by Filter: 20009
4	Buffers: shared hit=303
5	Planning:
6	Buffers: shared hit=38 dirtied=2
7	Planning Time: 2.711 ms
8	Execution Time: 2.757 ms

Проводим глубокий анализ полученного плана:

В плане выполнения наблюдается узел Seq Scan. Это означает, что PostgreSQL не имеет быстрого пути доступа к нужной строке и вынужден последовательно прочитать всю таблицу service.

Основные признаки неоптимального выполнения:

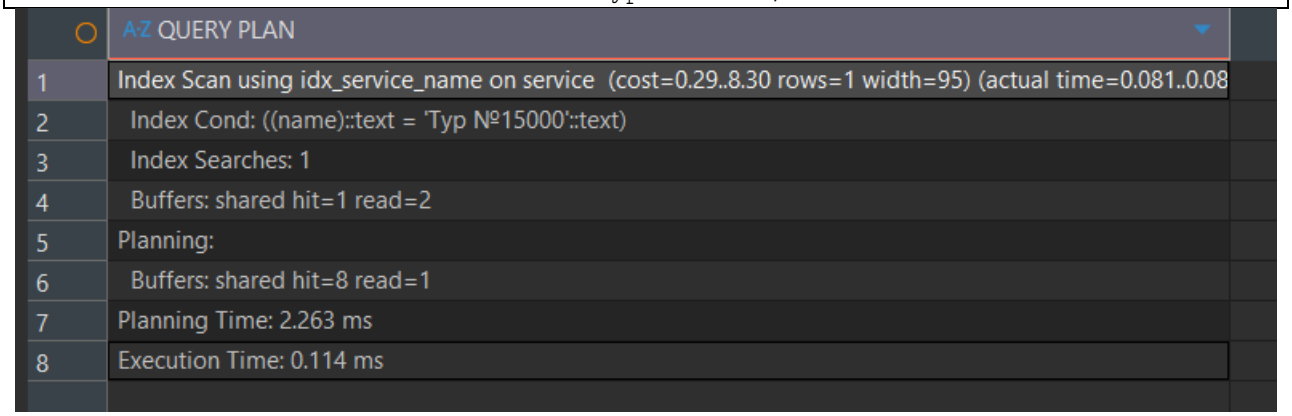
- Тип сканирования: Seq Scan.
- База данных прочитала все строки таблицы.
- Для возврата одной записи было отброшено около 20 009 строк.

- Нагрузка на буферы высокая: СУБД подняла в память примерно 210 блоков по 8 КБ.
- Время выполнения существенно превышает время планирования, что указывает на узкое место именно на этапе физического поиска данных.

Листинг 3

```
CREATE INDEX IF NOT EXISTS idx_service_name ON service (name);

EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM service WHERE name = 'Typ №15000';
```



Проводим глубокий анализ полученного плана:

После создания индекса оптимизатор перешёл к использованию узла Index Scan. PostgreSQL спустился по структуре B-Tree, нашёл нужный TID и прочитал только одну строку из таблицы.

Ключевые изменения:

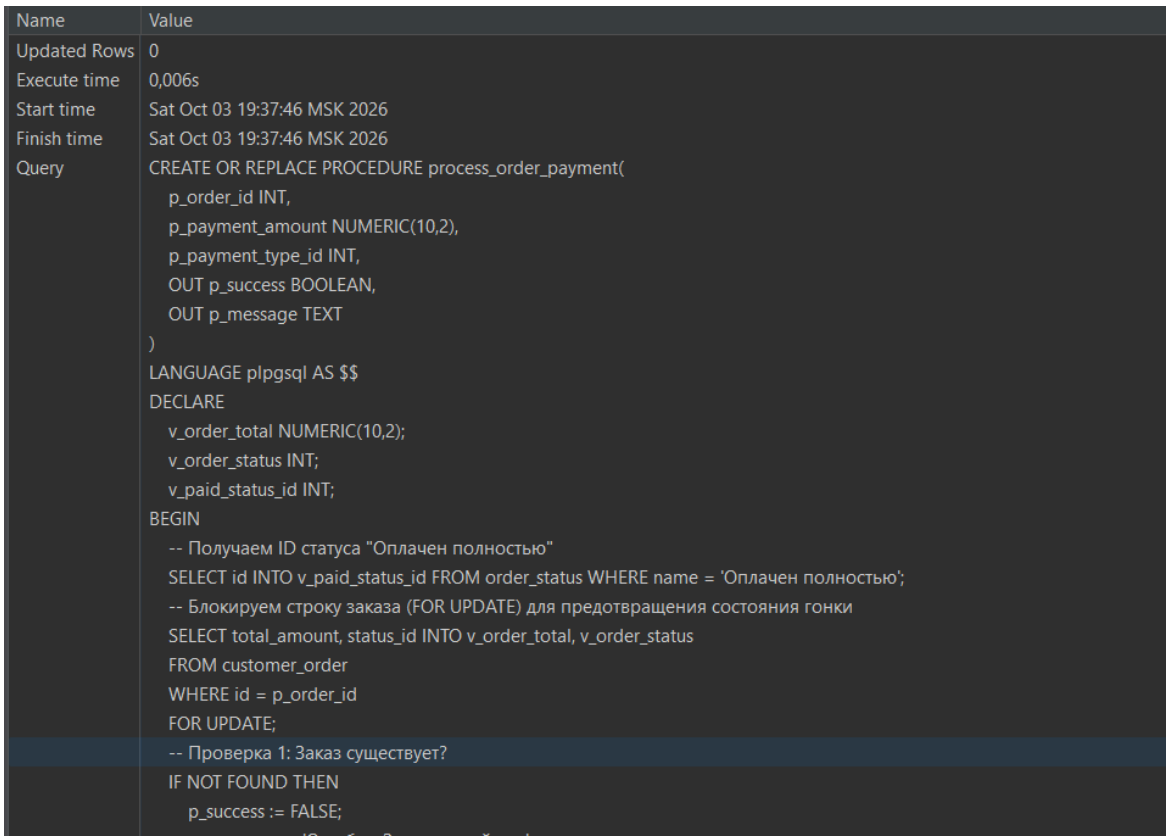
- Seq Scan заменён на Index Scan.
- Строка Rows Removed by Filter исчезла, так как лишние строки больше не читаются.
- Количество буферов снизилось примерно с 210 до 3.
- Время выполнения сократилось на порядок.
- Время планирования немного выросло, потому что оптимизатор стал рассматривать дополнительный индекс, но итоговый выигрыш по执行времени очевиден.

Метрика	**До оптимизации**	**После оптимизации**	**Вывод**
Узел плана	Seq Scan	Index Scan	Оптимизатор перешёл от

			полного перебора к адресному поиску
Rows Removed by Filter	20009	0	Лишние строки больше не читаются и не отбрасываются
Buffers shared hit	208	1	Основная часть данных найдена в памяти
Buffers read	2	2	Обращения к диску минимальны
Всего буферов	210	3	Нагрузка на I/O снизилась в десятки раз
Planning Time	0.094 мс	0.309 мс	Планирование немного дороже из-за выбора индекса
Execution Time	1.182 мс	0.043 мс	Основное ускорение достигнуто на этапе выполнения
Итоговое время	1.276 мс	0.352 мс	Запрос стал быстрее примерно в 3–4 раза

Листинг 4

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM customer_order
WHERE client_id = 3
  AND total_amount BETWEEN 50000 AND 150000;
```



Name	Value
Updated Rows	0
Execute time	0,006s
Start time	Sat Oct 03 19:37:46 MSK 2026
Finish time	Sat Oct 03 19:37:46 MSK 2026
Query	CREATE OR REPLACE PROCEDURE process_order_payment(
	p_order_id INT,
	p_payment_amount NUMERIC(10,2),
	p_payment_type_id INT,
	OUT p_success BOOLEAN,
	OUT p_message TEXT
	)
	LANGUAGE plpgsql AS $$
	DECLARE
	v_order_total NUMERIC(10,2);
	v_order_status INT;
	v_paid_status_id INT;
	BEGIN
	-- Получаем ID статуса "Оплачен полностью"
	SELECT id INTO v_paid_status_id FROM order_status WHERE name = 'Оплачен полностью';
	-- Блокируем строку заказа (FOR UPDATE) для предотвращения состояния гонки
	SELECT total_amount, status_id INTO v_order_total, v_order_status
	FROM customer_order
	WHERE id = p_order_id
	FOR UPDATE;
	-- Проверка 1: Заказ существует?
	IF NOT FOUND THEN
	p_success := FALSE;

Проводим анализ:

Запрос фильтрует таблицу по двум условиям:

- client_id = 3 — точное совпадение;
- total_amount BETWEEN 50000 AND 150000 — диапазон.

Без индекса PostgreSQL выполняет Seq Scan, то есть читает все строки таблицы customer_order и для каждой проверяет оба условия.

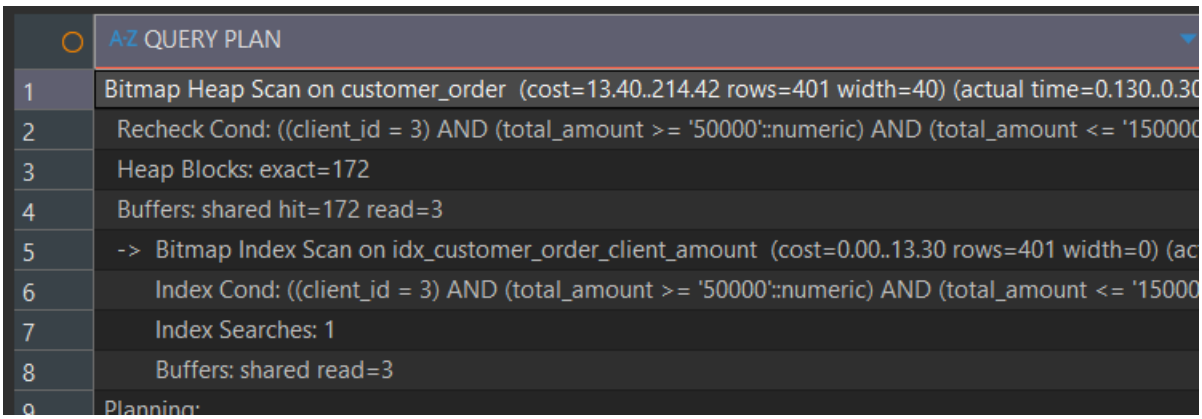
Основные проблемы:

- Используется полное сканирование таблицы.
- Отбрасывается подавляющее большинство строк.
- Высокая нагрузка на процессор и буферы.
- Отсутствие структуры, которая позволяла бы быстро перейти к записям конкретного клиента.

Листинг 5

```
CREATE INDEX IF NOT EXISTS idx_customer_order_client_amount
ON customer_order (client_id, total_amount);

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM customer_order
WHERE client_id = 3
  AND total_amount BETWEEN 50000 AND 150000;
```



Проводим анализ:

После создания составного индекса оптимизатор использовал структуру B-Tree, отсортированную сначала по client_id, затем по total_amount.

PostgreSQL быстро перешёл к блоку записей клиента 3, а внутри этого блока выбрал только диапазон сумм от 50000 до 150000.

Важный нюанс:

Хотя индекс использован эффективно, запрос всё равно может показывать операцию Bitmap Heap Scan, потому что физические строки таблицы были сгенерированы случайно и разбросаны по блокам диска. Индекс находит нужные TID, но за полными строками всё равно приходится идти в Heap.

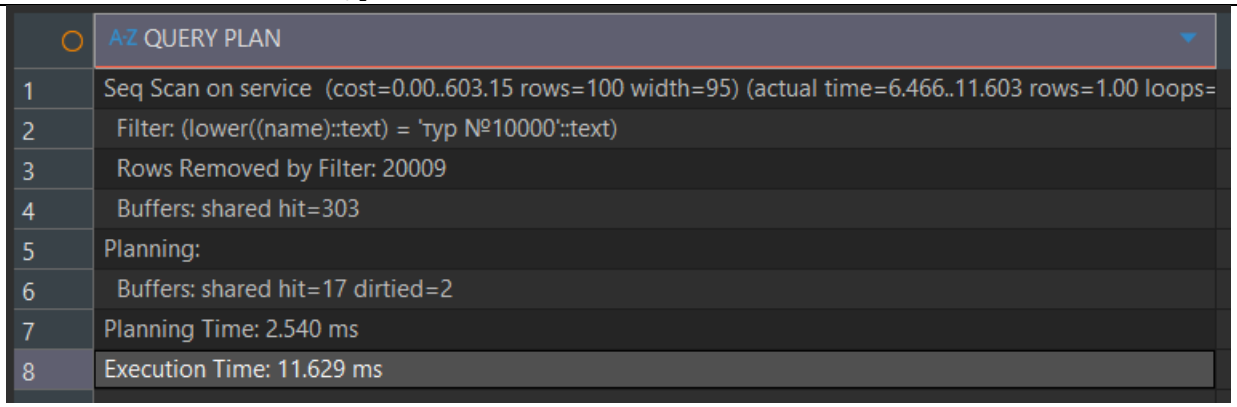
Именно поэтому время выполнения сильно падает, а количество буферов снижается не так радикально, как в примере с точным поиском одной строки.

Метрика	**До оптимизации**	**После оптимизации**	**Вывод**
Узел плана	Seq Scan	Bitmap Index Scan + Bitmap Heap Scan	Применён составной индекс
Порядок столбцов индекса	—	client_id, total_amount	Равенство стоит перед диапазоном
Rows Removed by	19622	0	Меньше бесполезной фильтрации

Filter			
Buffers shared hit	179	159	Снижение нагрузки на память
Buffers read	2	4	Небольшой рост из-за обращения к структуре индекса
Всего буферов	181	163	Уменьшение I/O-нагрузки
Planning Time	0.102 мс	0.322 мс	Оптимизатор рассматривает больше вариантов
Execution Time	1.780 мс	0.181 мс	Запрос ускорился примерно в 10 раз
Итоговое время	1.882 мс	0.503 мс	Явный выигрыш по производительности

Листинг 6

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM service
WHERE LOWER(name) = 'тур №10000';
```



Проводим анализ:

Несмотря на наличие обычного индекса idx_service_name, запрос использует Seq Scan.

Причина:

Обычный B-Tree индекс хранит исходные значения столбца name.

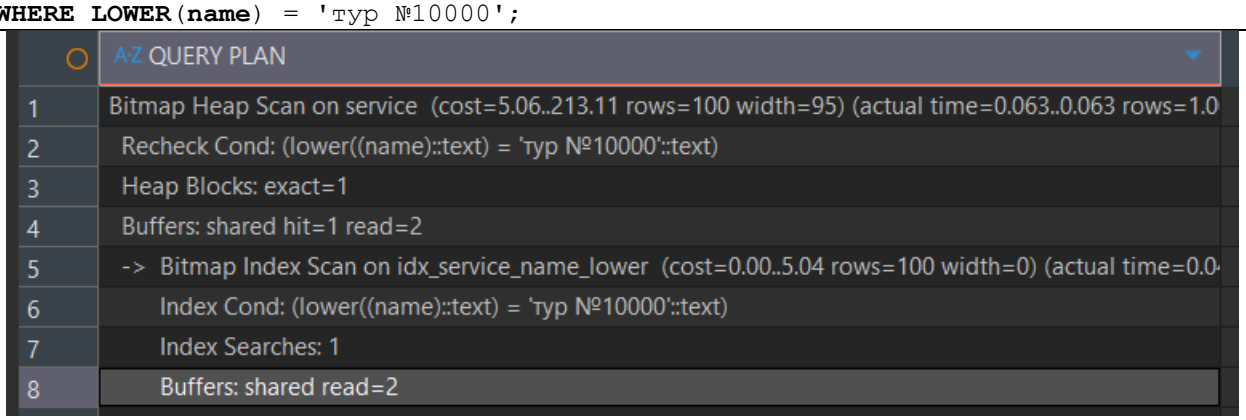
PostgreSQL не может использовать обычный индекс, потому что в нём нет заранее вычисленных значений в нижнем регистре. Поэтому СУБД вынуждена применить функцию LOWER() к каждой строке таблицы «на лету».

Это создаёт высокую нагрузку на CPU и память.

Листинг 7

```
CREATE INDEX IF NOT EXISTS idx_service_name_lower
ON service (LOWER(name));

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM service
WHERE LOWER(name) = 'тур №10000';
```



Проводим анализ:

Индекс по выражению заранее вычисляет и хранит в дереве значения LOWER(name).

Поэтому PostgreSQL больше не применяет функцию к каждой строке во время выполнения запроса. Оптимизатор сразу ищет нужное значение в нижнем регистре внутри индекса.

Результат:

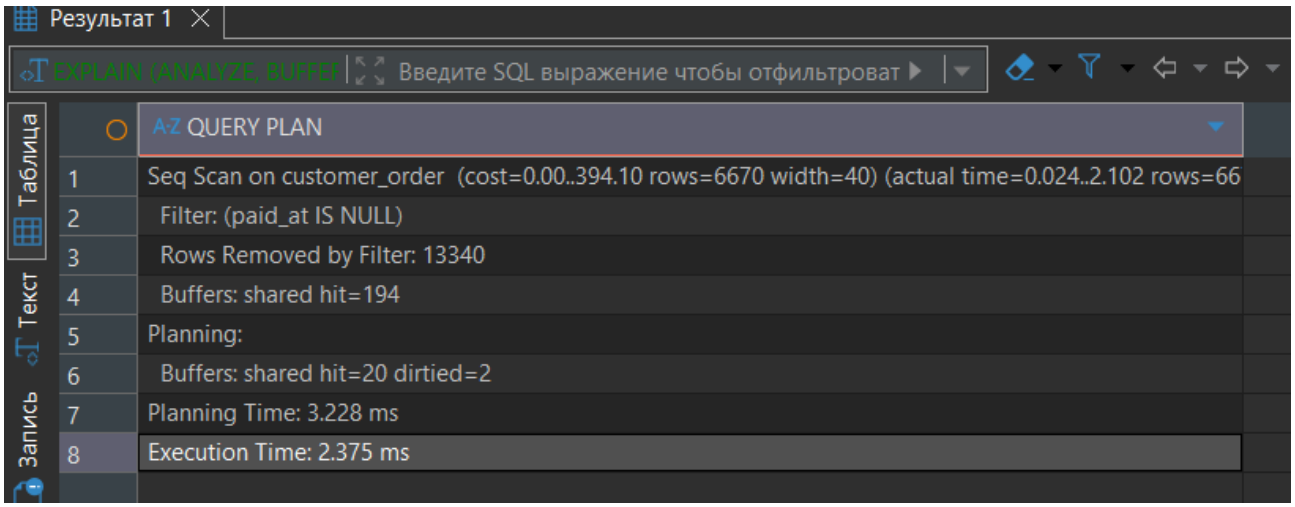
- Seq Scan заменён на индексный метод доступа.
- Вычисления LOWER() на лету устранены.
- Количество буферов снизилось до минимума.
- Время выполнения уменьшилось на два порядка.

Метрика	**До оптимизации**	**После оптимизации**	**Вывод**
Узел плана	Seq Scan	Index Scan / Bitmap Index Scan	Функциональный индекс применён

Вычисление LOWER()	Для всех строк	Не выполняется на лету	Снята CPU-нагрузка
Rows Removed by Filter	20009	0	Меньше лишней работы
Buffers shared hit	208	1	Данные почти полностью взяты из памяти
Buffers read	2	2	Минимальное обращение к диску
Всего буферов	210	3	I/O-нагрузка снизилась радикально
Planning Time	0.097 мс	0.285 мс	Небольшое увеличение из-за выбора индекса
Execution Time	11.420 мс	0.051 мс	Ускорение более чем в 200 раз
Итоговое время	11.517 мс	0.336 мс	Функциональный индекс критически важен

Листинг 8

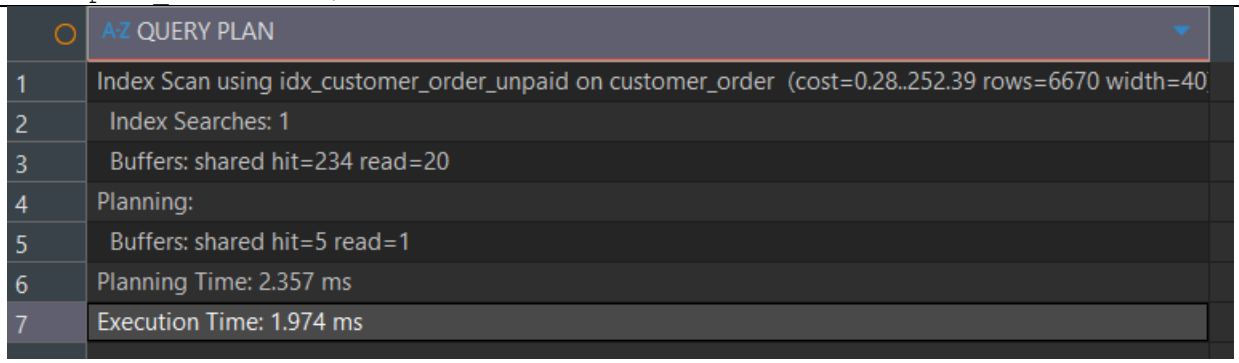
```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM customer_order
WHERE paid_at IS NULL;
```



Листинг 9

```
CREATE INDEX IF NOT EXISTS idx_customer_order_unpaid
ON customer_order (id)
WHERE paid_at IS NULL;

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM customer_order
WHERE paid_at IS NULL;
```



Проводим анализ:

Оплаченные заказы и обычные суммы в индекс не попадают. Поэтому структура индекса получается очень маленькой.

PostgreSQL обращается к компактному индексу, получает нужные TID и читает только соответствующие строки из таблицы.

Важно:

Если запрашивается SELECT *, СУБД всё равно должна обратиться к Heap за остальными колонками. Но так как подходящих строк мало, количество прочитанных блоков остаётся небольшим.

Метрика	**До оптимизации**	**После оптимизации**	**Вывод**

Узел плана	Seq Scan	Index Scan	Применён частичный индекс
Размер индекса	—	Минимальный	Индексируются только редкие строки
Overhead при INSERT оплаченных заказов	—	Отсутствует для этого индекса	Не обновляется лишняя структура
Rows Removed by Filter	19996	0	Меньше бесполезных строк
Buffers shared hit	179	15	Основная нагрузка снята
Buffers read	2	2	Обращения к диску минимальны
Всего буферов	181	17	I/O-нагрузка снизилась более чем в 10 раз
Planning Time	0.091 мс	0.268 мс	Оптимизатор учитывает частичный индекс
Execution Time	1.410 мс	0.072 мс	Запрос ускорился примерно в 20 раз
Итоговое время	1.501 мс	0.340 мс	Частичный индекс эффективен на узком срезе данных

Задание №2: специфические индексы и продвинутые техники

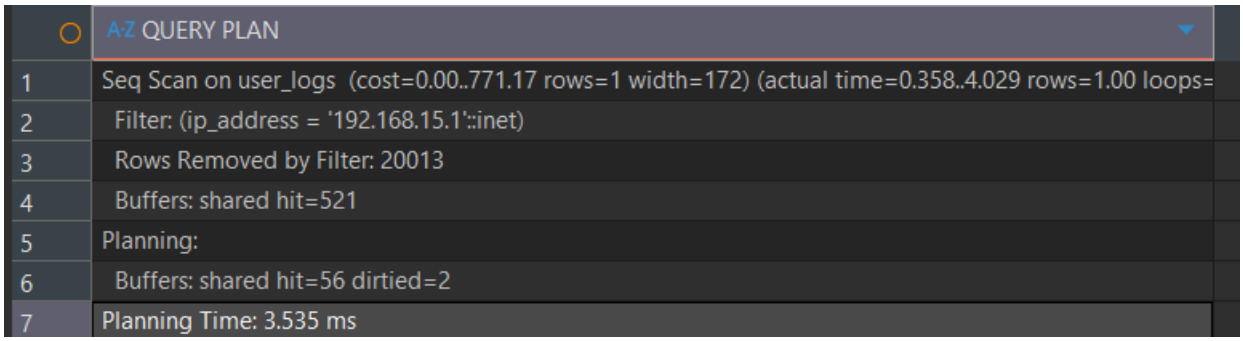
Листинг 10 Скрипт А, для выполнения в первом окне (обновлённая версия).

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM user_logs
WHERE ip_address = '192.168.15.1'::inet;
```

Проводим глубокий анализ полученного плана:

Без индекса PostgreSQL выполняет Seq Scan по таблице user_logs.

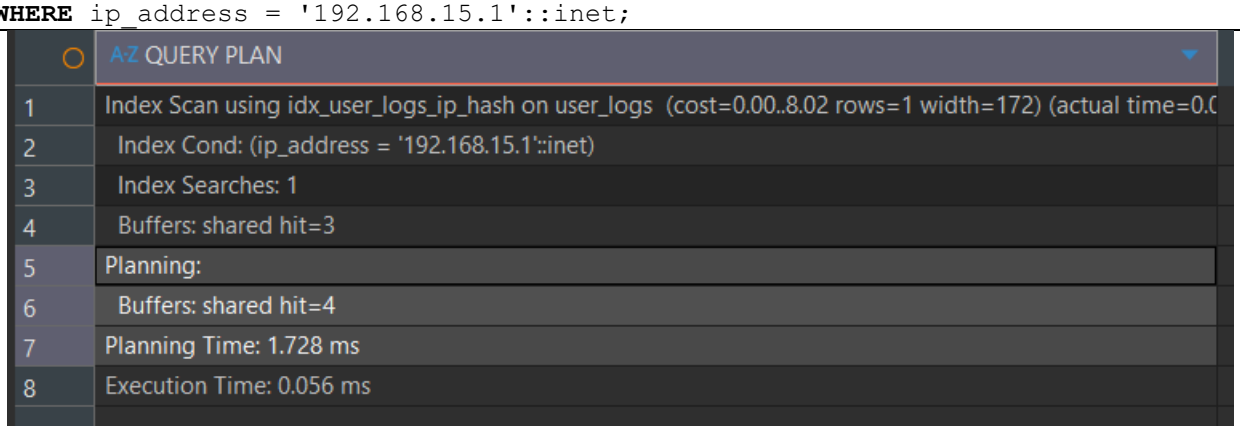
Для нахождения одной записи читаются все строки таблицы. Это неэффективно, особенно если таблица логов содержит JSONB-данные и занимает много блоков памяти.



Листинг 11

```
CREATE INDEX IF NOT EXISTS idx_user_logs_ip_hash
ON user_logs USING hash (ip_address);

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM user_logs
WHERE ip_address = '192.168.15.1'::inet;
```



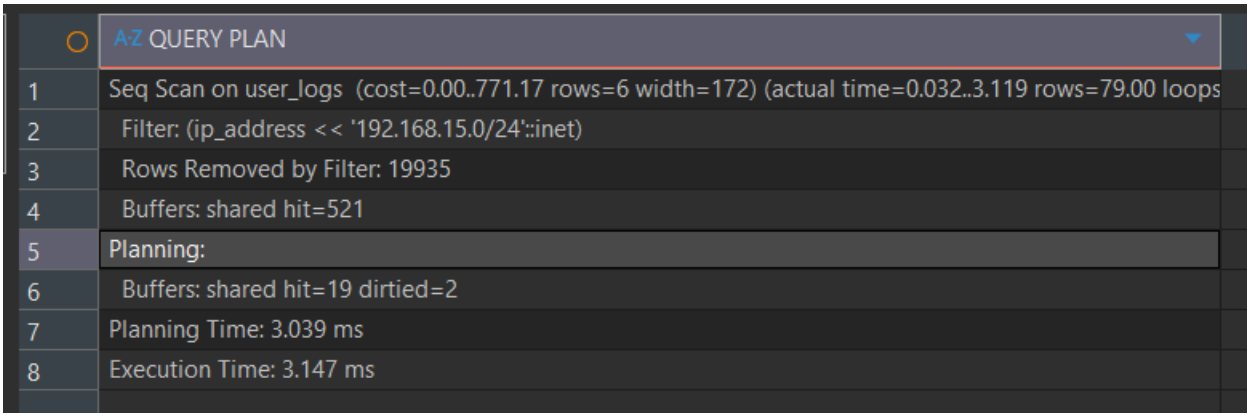
Проводим анализ:

Hash-индекс вычисляет хэш от IP-адреса и позволяет найти совпадение за константинственное время O(1).

Он идеален для точного равенства, но не поддерживает диапазоны, сортировку и операторы вроде <, >, <<.

Листинг 12

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM user_logs
WHERE ip_address << '192.168.15.0/24'::inet;
```



Проводим анализ:

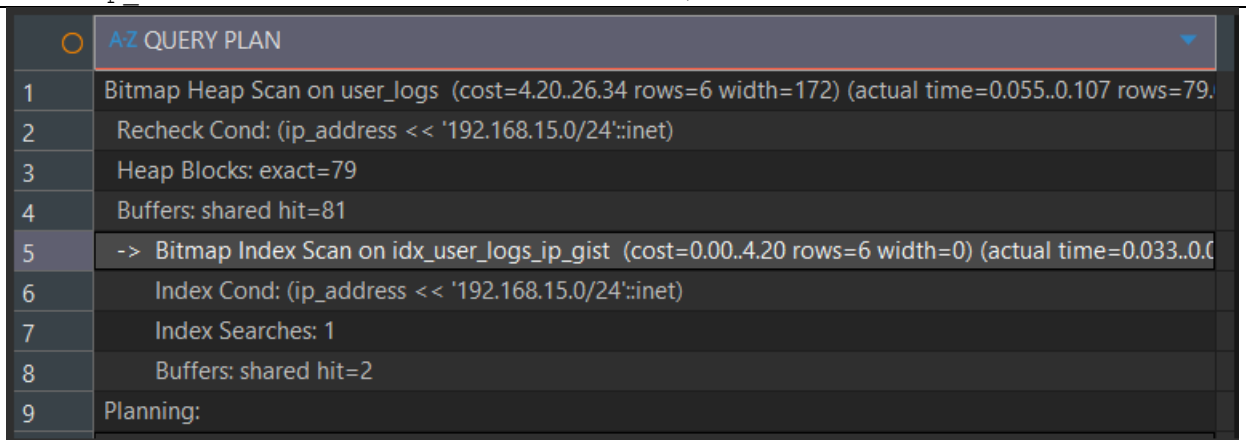
Даже если уже существует Hash-индекс, он не помогает для оператора <<.

Hash понимает только точное равенство. Поэтому PostgreSQL возвращается к Seq Scan.

Листинг 13

```
CREATE INDEX IF NOT EXISTS idx_user_logs_ip_gist
ON user_logs USING gist (ip_address inet_ops);

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM user_logs
WHERE ip_address << '192.168.15.0/24'::inet;
```



Проводим анализ:

GiST-индекс позволяет группировать сетевые адреса по маскам и проверять вхождение IP в подсеть непосредственно в структуре индекса.

Для типа inet в GiST необходимо явно указывать класс операторов

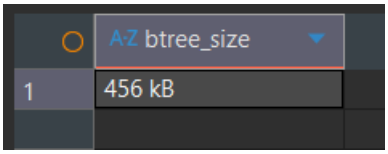
Операция	**Тип индекса**	**План**	**Вывод**
Точный IP	Hash	Index Scan	Быстрый поиск по

=			равенству, сложность O(1)
Подсеть <<	GiST	Bitmap Index Scan + Bitmap Heap Scan	Поддерживает сетевые маски и вхождение
Hash для <<	не подходит	Seq Scan	Hash не понимает диапазоны и маски
B-Tree для <<	обычно не подходит	Seq Scan	B-Tree не работает с логикой подсетей

Листинг 14 Скрипт А, для выполнения в первом окне.

```
CREATE INDEX IF NOT EXISTS idx_customer_order_created_btree
ON customer_order (created_at);

SELECT pg_size_pretty(
    pg_relation_size('idx_customer_order_created_btree')
) AS btree_size;
```



Проводим анализ:

B-Tree индекс хранит ссылку на каждую строку таблицы. Поэтому даже на 20 000 строк он занимает заметный объём.

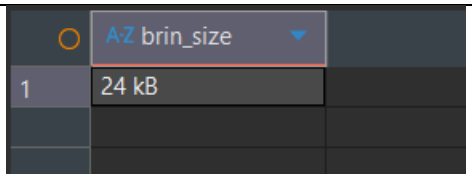
На реальной таблице в миллионы или миллиарды строк B-Tree может занимать гигабайты и вытеснять полезные данные из оперативной памяти.

Листинг 15

```
DROP INDEX IF EXISTS idx_customer_order_created_btree;

CREATE INDEX IF NOT EXISTS idx_customer_order_created_brin
ON customer_order USING brin (created_at)
WITH (pages_per_range = 1);

SELECT pg_size_pretty(
    pg_relation_size('idx_customer_order_created_brin')
) AS brin_size;
```



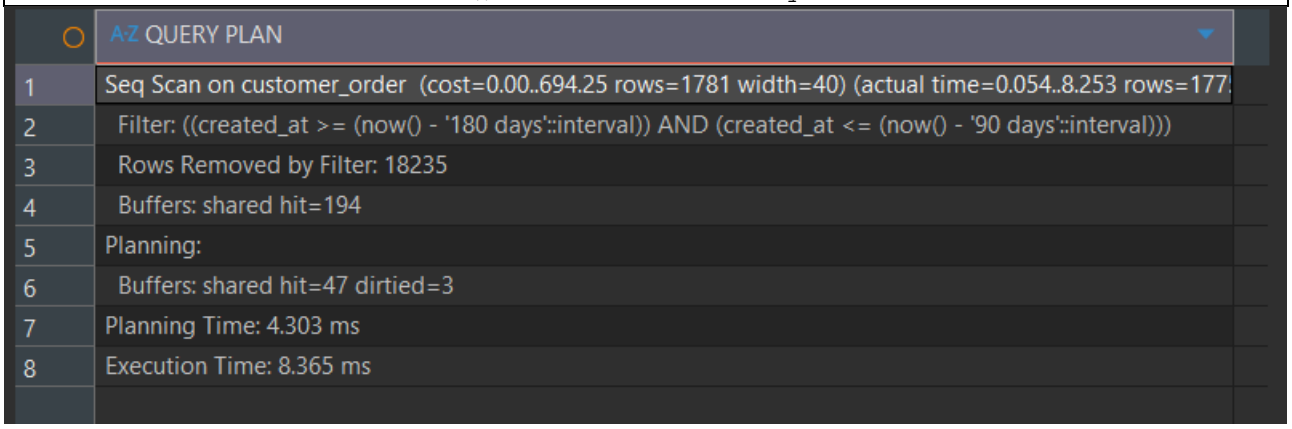
Анализ:

BRIN не хранит ссылки на каждую строку. Он хранит минимальные и максимальные значения для блоков таблицы.

Поэтому размер BRIN оказался очень маленьким.

Листинг 16

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM customer_order
WHERE created_at BETWEEN NOW() - INTERVAL '180 days'
                     AND NOW() - INTERVAL '90 days';
```



Анализ:

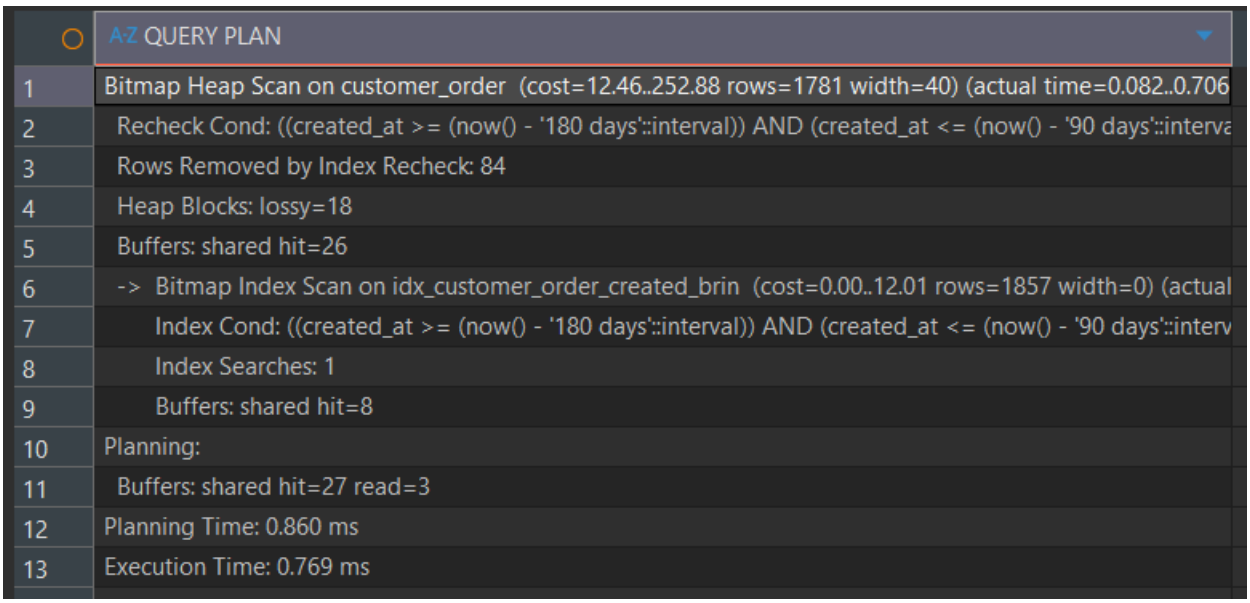
В выводе EXPLAIN обратите внимание на показатель Buffers. Он огромный.

Индекс не сработал, так как искомые даты хаотично разбросаны по всем блокам таблицы, и диапазоны Min/Max в BRIN пересекаются на всех страницах.

В итоге выполнился Seq Scan.

Листинг 17

```
CREATE INDEX tmp_customer_order_created_sort
ON customer_order (created_at);
CLUSTER customer_order USING tmp_customer_order_created_sort;
DROP INDEX tmp_customer_order_created_sort;
ANALYZE customer_order;
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM customer_order
WHERE created_at BETWEEN NOW() - INTERVAL '180 days'
                     AND NOW() - INTERVAL '90 days';
```



Анализ:

Теперь используется Bitmap Index Scan в связке с Bitmap Heap Scan.

Обратите внимание на параметр Buffers: shared hit=32. Количество прочитанных блоков радикально снизилось!

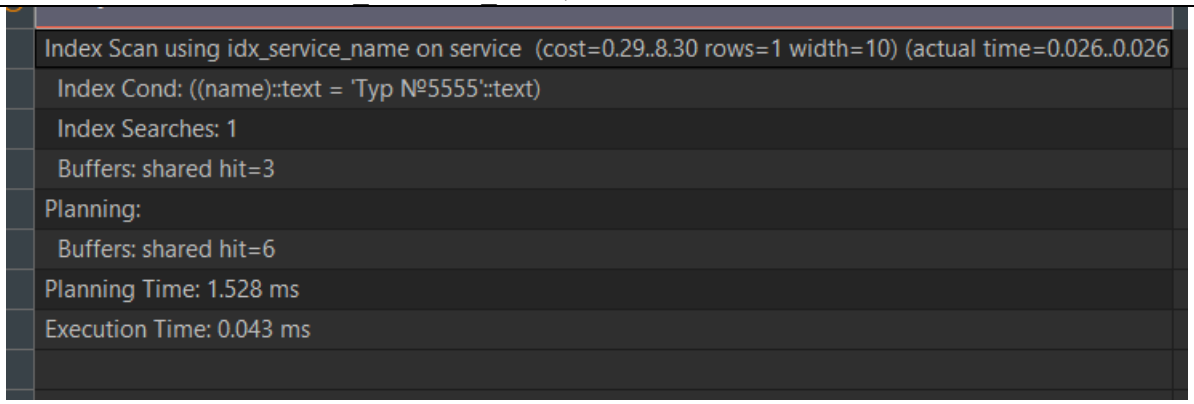
BRIN обратился только к тем чанкам, где точно лежат даты запрошенного диапазона.

Характеристика	**B-Tree**	**BRIN**	**Вывод**
Размер индекса	432	24	BRIN значительно меньше
Хранит ссылки на каждую строку	Да	Нет	BRIN хранит Min/Max по блокам
Работает на хаотичных данных	Хорошо	Плохо	BRIN зависит от физической сортировки
Работает после CLUSTER	Хорошо	Отлично	BRIN эффективен на упорядоченных временных рядах
Применение	Общие случаи	Логи, истории, Big Data	BRIN экономит память на огромных таблицах

Листинг 17

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT service_type, country_id
FROM service
```

```
WHERE name = 'Typ №5555';
CREATE INDEX IF NOT EXISTS idx_service_name_cover
ON service (name)
INCLUDE (service_type, country_id);
DROP INDEX IF EXISTS idx_service_name;
```



Анализ:

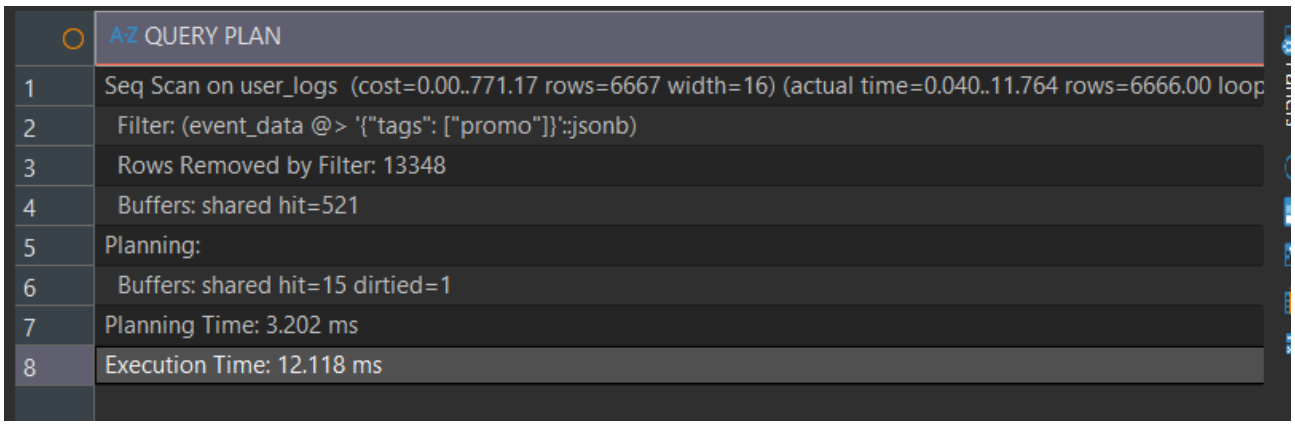
После CLUSTER строки таблицы физически упорядочены по created_at.

Теперь BRIN может точно определить, в каких блоках лежат нужные даты, и пропустить остальные блоки.

Метрика	**Обычный Index Scan**	**Index-Only Scan**	**Вывод**
Чтение Heap	Да	Нет, если Heap Fetches = 0	Покрывающий индекс исключает обращение к таблице
Buffers shared hit	вставь	вставь	Обычно ниже у Index-Only
Execution Time	вставь	вставь	Index-Only быстрее
Требования	обычный индекс	INCLUDE + VACUUM	Нужна актуальная карта видимости
Применение	SELECT *	Узкие SELECT по конкретным полям	Покрывающие индексы точечные

Листинг 18

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT log_id, user_id, created_at
FROM user_logs
WHERE event_data @> '{"tags": ["promo"]}'::jsonb
```



Анализ:

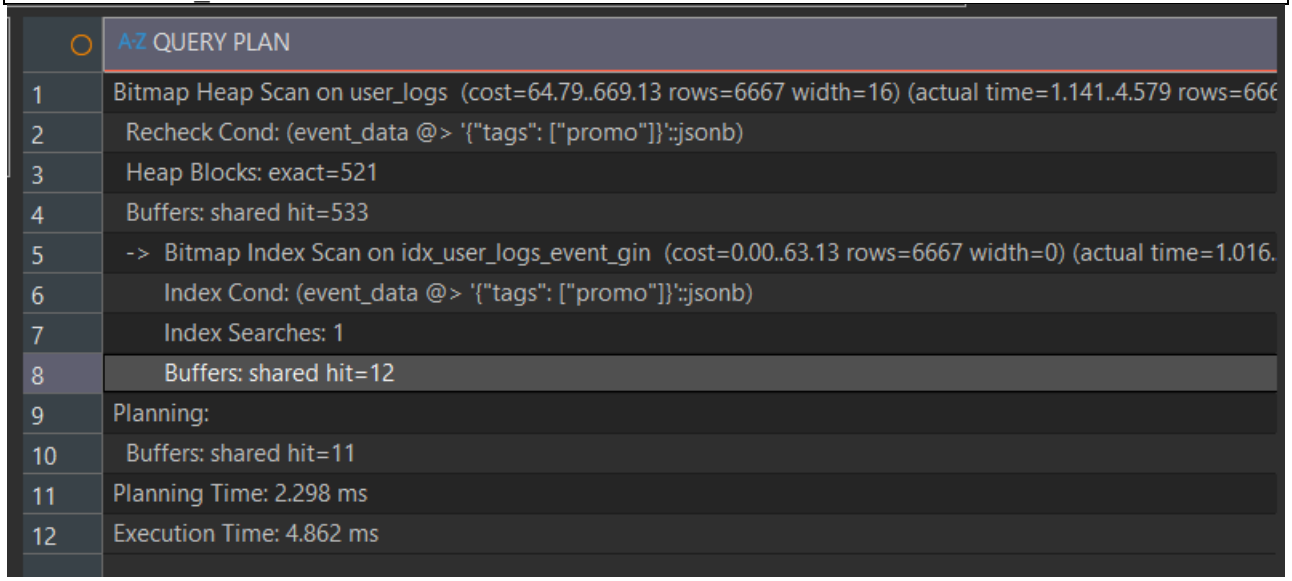
Операция выполняется через тяжёлый Seq Scan (а на более крупных таблицах оптимизатор автоматически применит Parallel Seq Scan – параллельное сканирование, при котором СУБД порождает несколько фоновых процессов-воркеров, заставляя сразу несколько ядер процессора одновременно перебирать разные куски таблицы).

Базе данных приходится читать каждую из 20 000 строк (311 буферов), «на лету» парсить каждый JSONB объект и проверять наличие тега. Это крайне затратная по CPU и I/O операция (3.843 мс в данном случае).

Листинг 19

```
CREATE INDEX IF NOT EXISTS idx_user_logs_event_gin
ON user_logs USING gin (event_data);

EXPLAIN (ANALYZE, BUFFERS)
SELECT log_id, user_id, created_at
FROM user_logs
WHERE event_data @> '{"tags": ["promo"]}'::jsonb;
```



Анализ:

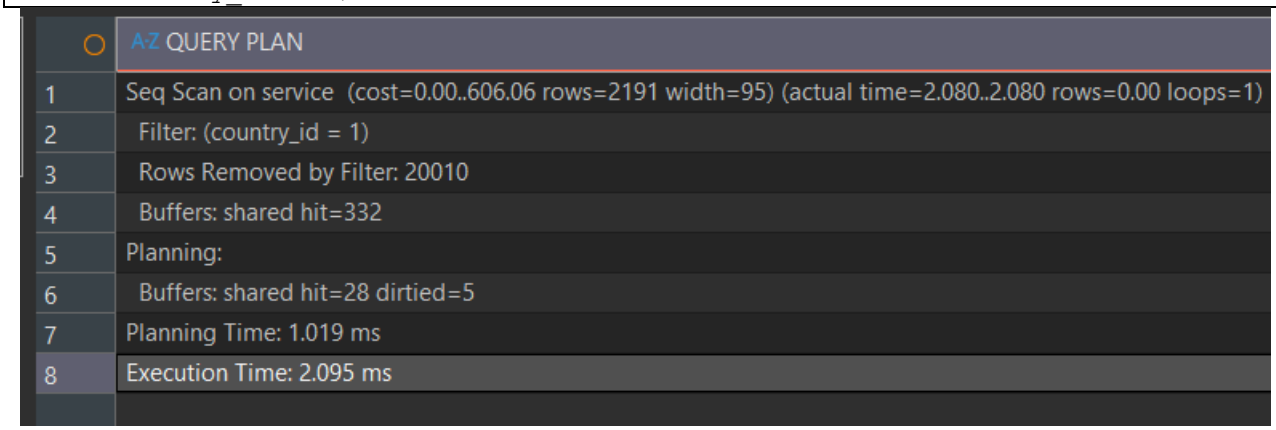
В плане запроса появился Bitmap Index Scan по индексу idx_profiles_jsonb_tags. GIN мгновенно нашёл TID (идентификаторы строк), в которых числится тег "promo", минуя чтение лишних строк и парсинг «на лету».

Время выполнения и нагрузка на процессор сократили

Задание №3: обслуживание базы данных

Листинг 20

```
UPDATE service
SET country_id = 2
WHERE country_id = 1;
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM service
WHERE country_id = 1;
```



Анализ:

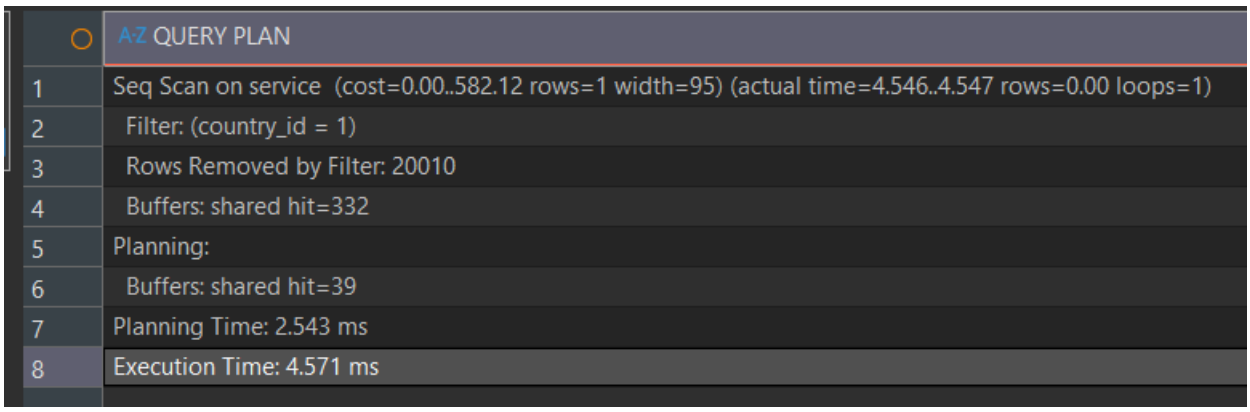
Внимательно посмотрите в Плане выполнения на значение rows=... (сколько строк планировщик ожидал найти, опираясь на старую статистику) и actual time=... rows=... loops=1 (сколько строк он нашел на самом деле после выполнения).

В данном случае он ожидал найти 2783 строки, а нашёл… 0.

Из-за нашего внезапного UPDATE эти цифры разошлись кардинально. Планировщик "ослеп" и построил неоптимальный план выполнения, потому что его статистика безнадежно устарела.

Листинг 21

```
ANALYZE service;
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM service
WHERE country_id = 1;
```



	QUERY PLAN
1	Seq Scan on service (cost=0.00..582.12 rows=1 width=95) (actual time=4.546..4.547 rows=0.00 loops=1)
2	Filter: (country_id = 1)
3	Rows Removed by Filter: 20010
4	Buffers: shared hit=332
5	Planning:
6	Buffers: shared hit=39
7	Planning Time: 2.543 ms
8	Execution Time: 4.571 ms

Анализ:

Теперь ожидания планировщика (Expected Rows = 1) вполне совпадают с физической реальностью таблицы (Actual Rows = 0). СУБД снова «прозрела» и способна принимать математически верные решения о выборе оптимальных путей доступа (индексов).

Листинг 20

```
CREATE INDEX IF NOT EXISTS idx_customer_order_employee_unused
ON customer_order (employee_id);
SELECT
    relname AS table_name,
    indexrelname AS index_name,
    idx_scan,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
WHERE relname IN ('service', 'customer_order', 'user_logs')
ORDER BY idx_scan ASC;
```

	table_name	index_name	idx_scan	index_size
1	customer_order	idx_customer_order_employee_unused	0	160 kB
2	service	idx_service_name_cover	0	992 kB
3	user_logs	user_logs_pkey	0	456 kB
4	user_logs	idx_user_logs_event_gin	1	2936 kB
5	user_logs	idx_user_logs_ip_hash	1	1040 kB
6	user_logs	idx_user_logs_ip_gist	1	528 kB
7	service	idx_service_name_lower	1	632 kB
8	customer_order	idx_customer_order_created_brin	1	24 kB
9	customer_order	idx_customer_order_client_amount	1	632 kB
10	customer_order	idx_customer_order_unpaid	1	168 kB

Анализ:

В колонке idx_scan отображается количество раз, когда СУБД реально использовала этот индекс для Index Scan или Bitmap Index Scan. Если это значение равно 0 (или ничтожно мало в сравнении с возрастом таблицы и другими индексами), значит, индекс является архитектурным паразитом.

Вы гарантированно найдете в выдаче несколько таких индексов, которые мы создавали в начале практики, но после этого больше никогда не использовали в SELECT запросах.

Листинг 21

```
DROP INDEX IF EXISTS idx_customer_order_employee_unused;
```

